

Introducción a la materia

Tecnología Digital VI

Licenciatura en Tecnología Digital - UTDT

Plantel docente



Federico Pousa



Gerardo Del Toro



Martín Sinnona

Horarios

Teóricas:

- Lunes 13:45 a 15:20 (SVE4)
- Jueves 13:45 a 15:20 (SVE4)
- Excepciones: 5/3, 24/3, 2/4, 25/5, 22/6

Prácticas:

- Martes: 13:45 a 15:20 (SV404)
- Viernes 13:45 a 15:20 (SV302)
- Excepciones: 23/3, 3/4

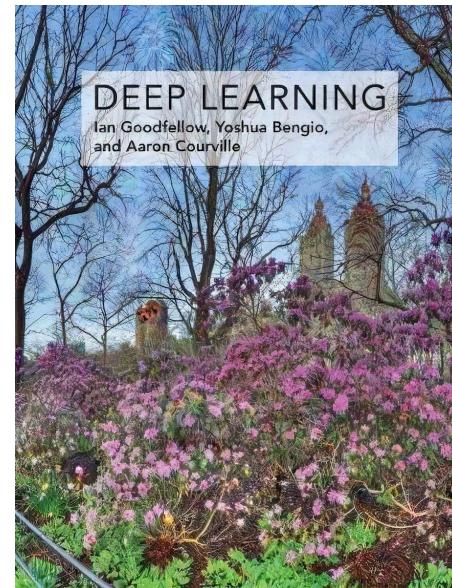
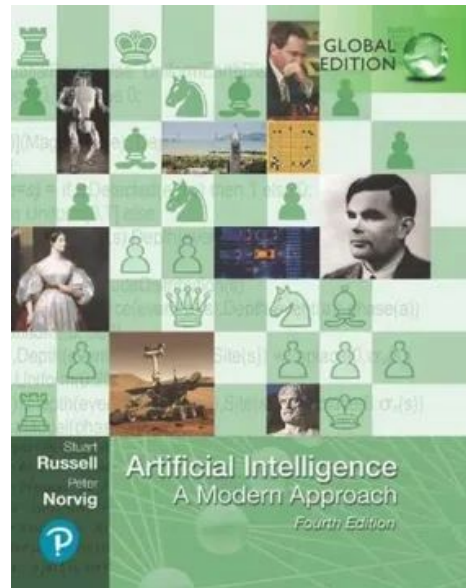
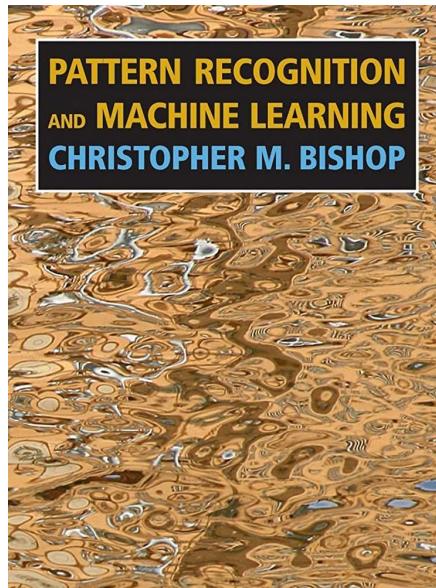
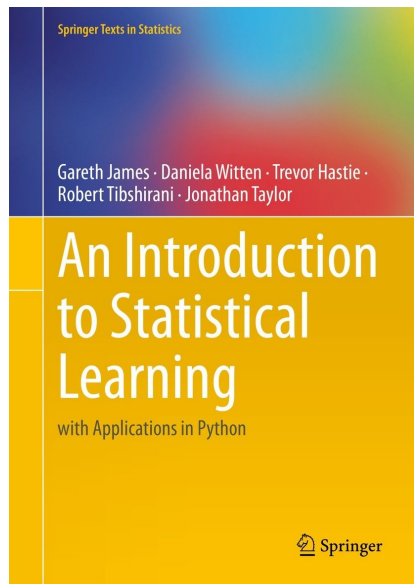
Formalidades

Régimen de aprobación:

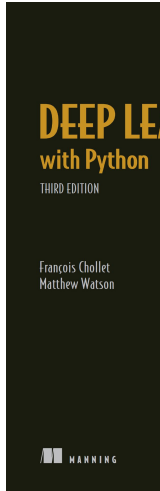
- 2 parciales individuales. Cada uno se aprueba con 60 puntos sobre 100.
- 4 trabajos prácticos grupales. Grupo de 3 personas.
- Cada instancia de evaluación tiene su instancia de recuperación.
- Se podrán recuperar un máximo de 3 instancias de evaluación.

Toda comunicación escrita se hará por el campus.

Bibliografía Teórica

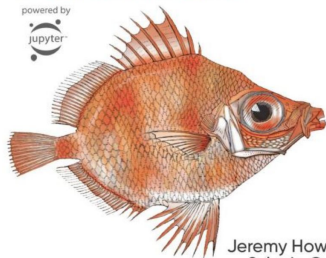


Bibliografía Práctica



Deep Learning for Coders with fastai & PyTorch

AI Applications Without a PhD



Jeremy Howard &
Sylvain Gugger
Foreword by Soumith Chintala

O'REILLY

Designing Machine Learning Systems

An Iterative Process
for Production-Ready
Applications



Chip Huyen

EXPERT INSIGHT

Sebastian Raschka
& Vahid Mirjalili

Python Machine Learning

Machine Learning and Deep Learning
with Python, scikit-learn, and TensorFlow

Second Edition - Fully revised and updated



Packt

O'REILLY

Python Data Science Handbook

Essential Tools for Working with Data

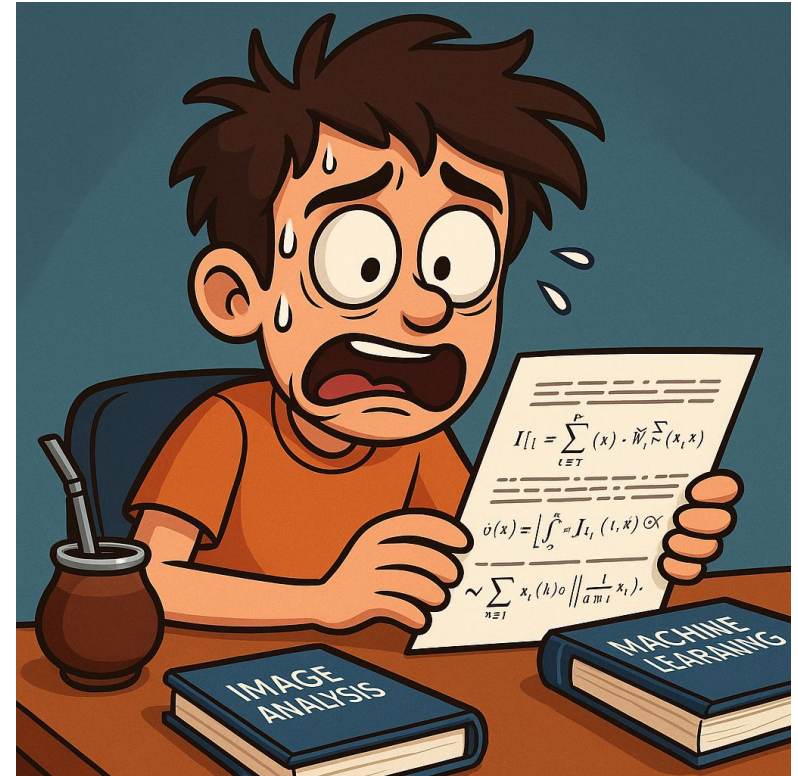


Jake VanderPlas

Second
Edition

Bibliografía – Papers

Sin miedo a leer papers. A veces son una inversión más grande de tiempo y además carecen de la estructura organizadora de conceptos como un buen libro, pero suelen ser la expresión más pura de los modelos que vamos a ver.



Introducciones, definiciones y más

¿Qué es “Inteligencia Artificial”?



Agustín Gennoni

@agustingennoni

...

Todo lo que tenía cheddar luego pasó a tener pistacho, y todo lo tenía pistacho ahora tiene inteligencia artificial.

[Translate post](#)

8:20 PM · Jul 27, 2024 · **451.6K** Views

¿Qué es la Inteligencia Artificial?

“We call ourselves Homo Sapiens because our intelligence is so important to us. For thousands of years, we have tried to understand how we think and act. [...] The field of Artificial Intelligence, or AI, is concerned with not just understanding but also building intelligente entities-machines that can compute how to act effectively and safely in a wide variety of novel situations.”

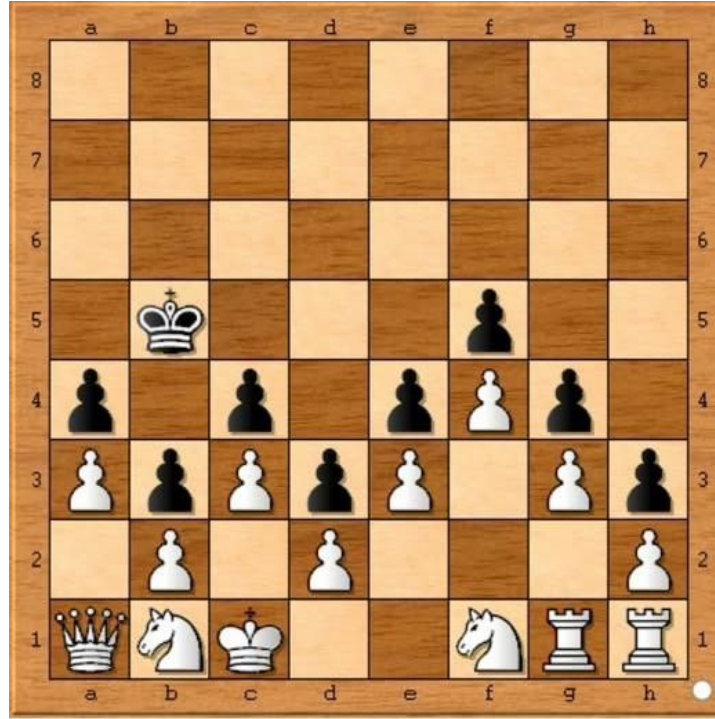
¿Qué es la Inteligencia Artificial?

¿Qué es la inteligencia? ¿Qué significa que una computadora sea “inteligente”?

Primera idea, Test de Turing:

Una computadora pasa el Test de Turing si, después de un ida y vuelta de varias preguntas, una persona no puede distinguir si las respuestas vienen de una persona o de una computadora.

¿Qué es la Inteligencia Artificial?



P A Lamford 1981 White to Play and Win. ~~Still not solved by Computer~~

¿Qué es la Inteligencia Artificial?

Una versión que prevaleció en la historia de la disciplina es la del *Agente racional*.

“In a nutshell, AI has focused on the study and construction of agents that do the right thing.”

Es lo que se conoce como el modelo estándar en el cual “qué es lo correcto” debe ser provisto al agente.

Es un modelo acotado, dado que “lo correcto” es claro en ciertos problemas acotados y muy poco claro en problemas del mundo real.

Se llama Value Alignment Problem a alinear los objetivos y valores humanos con los objetivos y valores que se le da al agente. De nuevo, algo muy complejo de definir y conseguir en el mundo real (un tema muy vigente con los actuales LLMs)

Lo que no vamos a ver

Inteligencia Artificial es un campo de técnicas mucho más abarcativo, pero no da el tiempo para charlar de todo:

- Agentes lógicos (lógica de primer orden, inferencia).
- Representación de conocimiento.
- Planeamiento automático.
- Razonamiento probabilístico.
- Sistemas Expertos.

Lo que sí vamos a ver

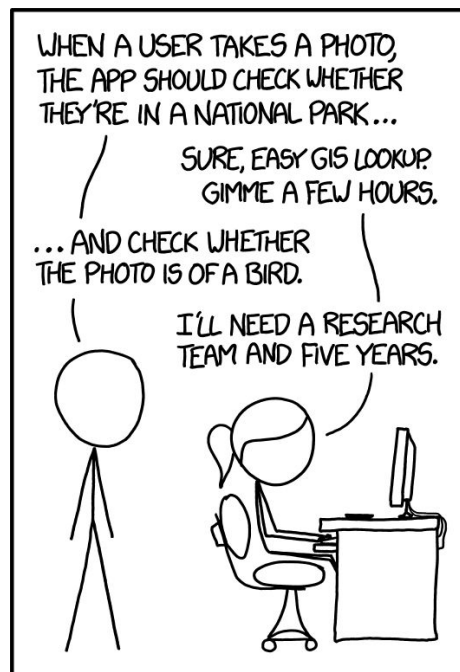
Si bien las demás técnicas se siguen utilizando para varios problemas, el campo evolucionó de el “conocimiento experto pasado a código” a el aprendizaje automático a partir de los datos disponibles.

No vamos a decir “Un experto hace A y B en esta situación, considéralas y evalúalas”. Más bien va a ser un “En todos estos datos, esto es lo que te debería dar, aprendé a que te de eso”.

Estructura de la materia

- Primera parte (Machine Learning):
 - Aprendizaje automático “clásico” (tabular/estructurado).
 - En general Aprendizaje Supervisado pero también algo de No Supervisado).
 - Métodos Lineales. Métodos Basados en Árboles. Ensamblados.
 - PCA/Clustering.
 - Ciclo de vida de un problema real de Machine Learning.
 - Data Acquisition. Feature Selection. Feature Engineering. Model Selection. Métricas. Puesta en producción.
- Segunda parte (Deep Learning):
 - Aprendizaje Profundo (no tabular / no estructurado, visión, texto).
 - Redes Neuronales. CNNs. RNNs. Arquitecturas específicas.
 - Problemas y aplicaciones actuales.

Machine Learning



IN CS, IT CAN BE HARD TO EXPLAIN
THE DIFFERENCE BETWEEN THE EASY
AND THE VIRTUALLY IMPOSSIBLE.

¿Cómo resolvemos esto?



¿Cómo resolvemos esto?



Machine Learning

*“Field of study that gives computers the ability to learn without being explicitly programmed” - Arthur Samuel, **1959***

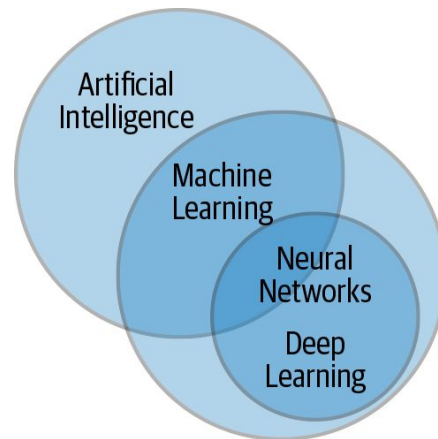
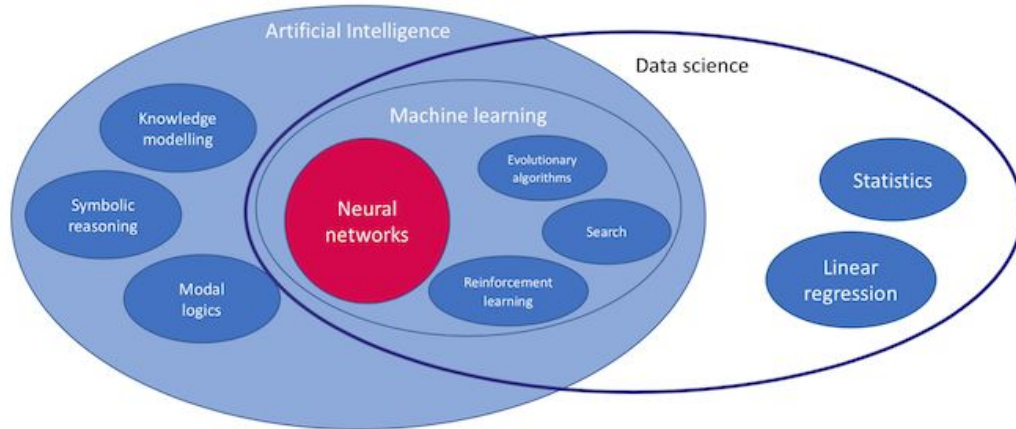
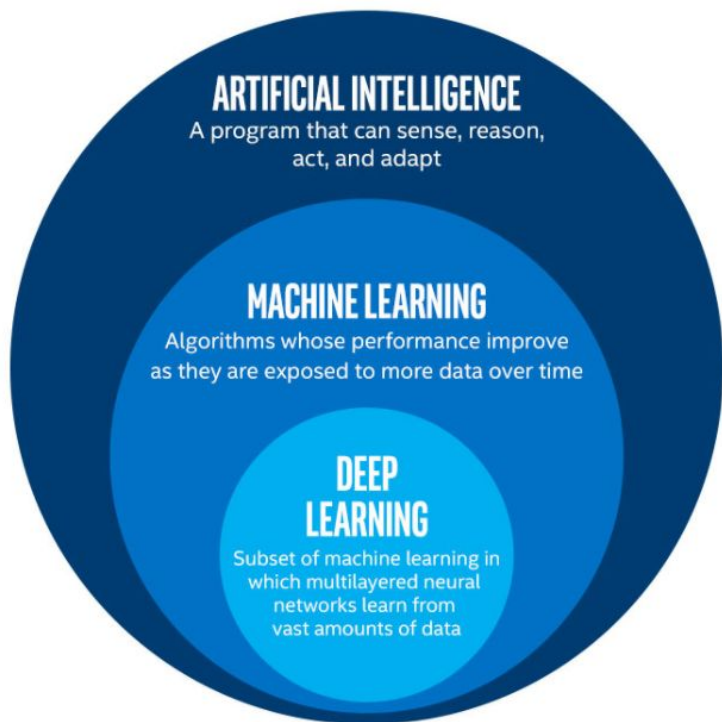


Machine Learning

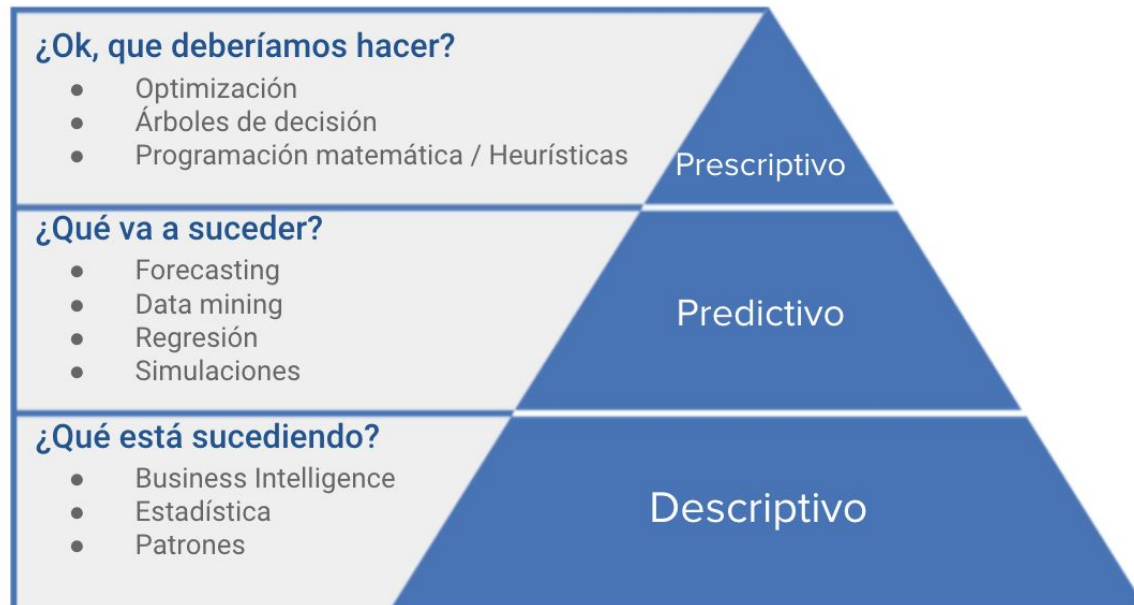
"A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P if its performance at tasks in T , as measured by P , improves with experience E ." - Tom Mitchell, 1997



Definiciones



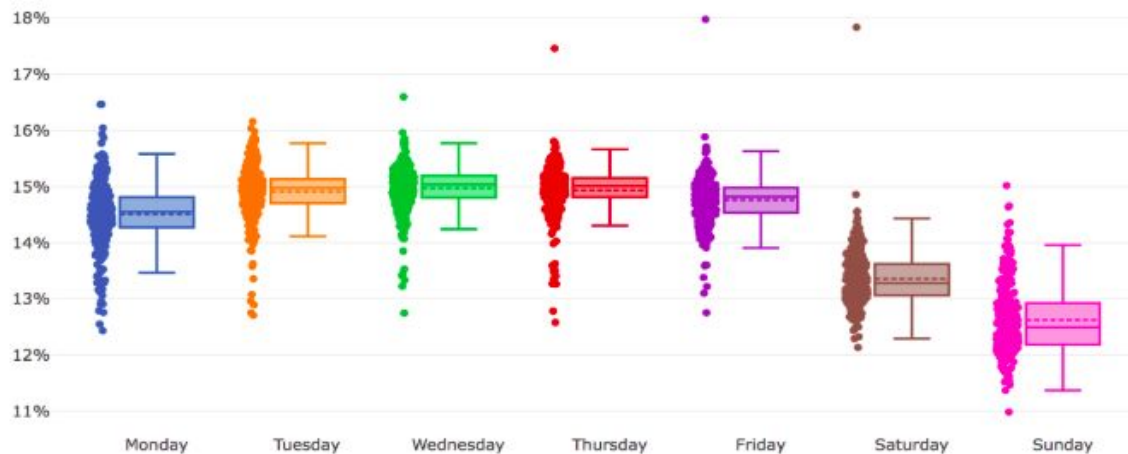
¿Qué podemos resolver?



Aplicaciones – Forecasting de clientes

Objetivo: predecir la demanda de clientes de grúas de autos para ordenar la operatoria.

Posiblemente me importa más predecir que entender.



Aplicaciones – Churn Prediction

Objetivo: Retener “usuarios” en un determinado servicio o producto.

Posiblemente nos interese mucho más entender las causas que la predicción en sí (que igual va a ser útil).



It costs five times as much to attract a new customer, than to keep an existing one



Aplicaciones – Fraud detection

Objetivo: detectar transacciones fraudulentas.

En este caso la necesidad es casi 100% de predicción y prácticamente nada de entendimiento.

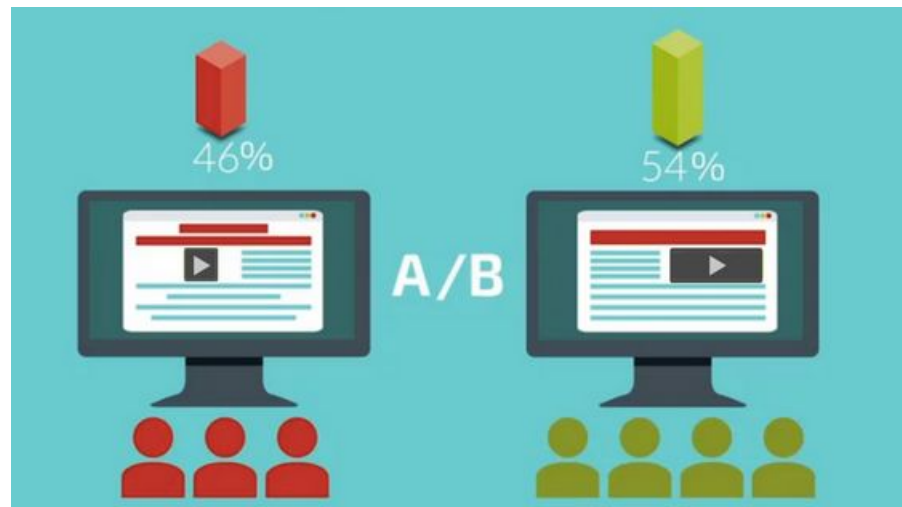


Aplicaciones – A/B Testing?

En un A/B testing la idea es que los grupos no sean distinguibles en su armado, así obtenemos conclusiones generalizables.

Nos inventamos un problema donde la variable a predecir es la pertenencia a cada grupo. Si un clasificador a ciegas “tiene señal” entonces no está bien dividido.

Hagamos una demo.



Aplicaciones

- Credit scoring.
- Despacho de pacientes.
- Segmentación de clientes.
- Sistemas de recomendación.
- Forecasting de demanda.
- Sentiment analysis.
- Clasificación de imágenes.
- Detección de objetos.
- ...

Tipos de ML*

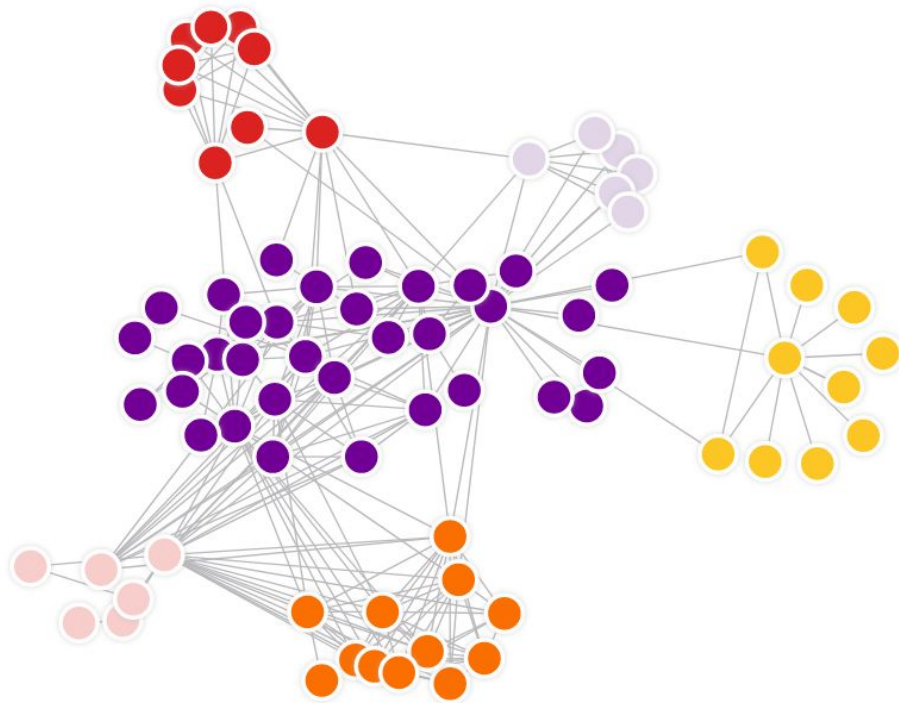
- Supervised Learning (Aprendizaje Supervisado): Los datos que utilizamos cuentan con *features* y *labels*. El objetivo es conseguir algún tipo de función que nos lleve de los features a las labels (que pueden ser clases o valores continuos).
- Unsupervised Learning: Los datos que utilizamos cuentan con features, no hay ninguna label a explicar. El objetivo es encontrar estructuras y patrones dentro de los datos disponibles.
- Reinforcement Learning: Los datos no se piensan como algo estático con features y labels, sino que son problemas en los que vamos tomando acciones y eso nos va llevando a output deseables o no deseables.

Supervised Learning – Ejemplo

Casi todos los mencionados en estas slides! Solo es cuestión de pensar cuáles son las features y cuál es la variable a predecir.

Unsupervised Learning – Ejemplo

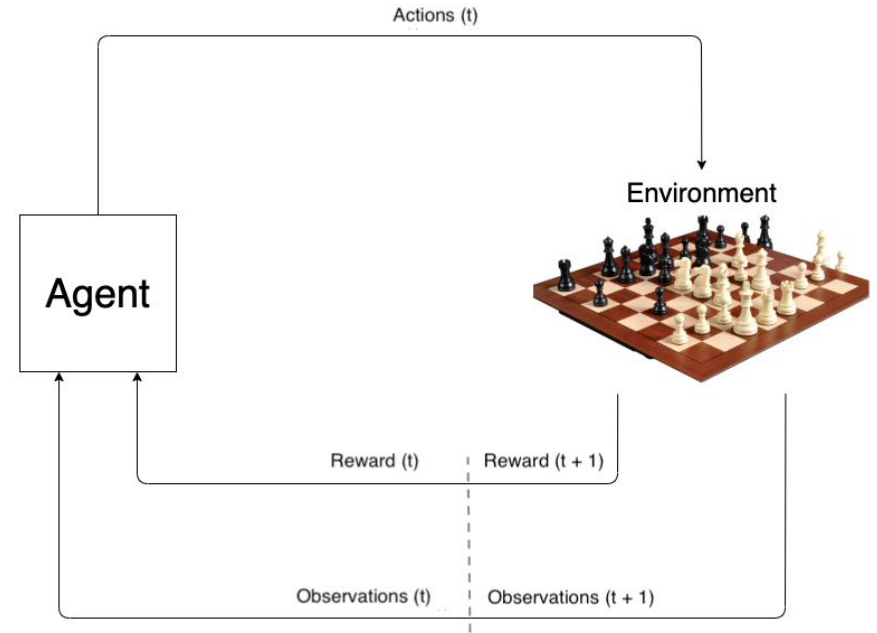
La clusterización es un ejemplo clásico de unsupervised learning. Por ejemplo se utiliza para analizar la interacción de las personas en redes sociales y así poder identificar diferentes grupos.



Reinforcement Learning – Ejemplo

Existen un montón de problemas que no podemos etiquetar de antemano si algo es bueno o malo. Debemos ir tomando acciones y viendo si eso nos lleva a un estado deseable o a uno indeseable.

Ejemplos concretos pueden ser juegos como el ajedrez o el go, aprender a manejar, aprender a balancear algo, etc.



Reinforcement Learning – Recomendaciones



<https://www.youtube.com/watch?v=WXuK6gekU1Y>

RESEARCH

AI achieves silver-medal standard
solving International Mathematical
Olympiad problems

25 JULY 2024

AlphaProof and AlphaGeometry teams

[Share](#)



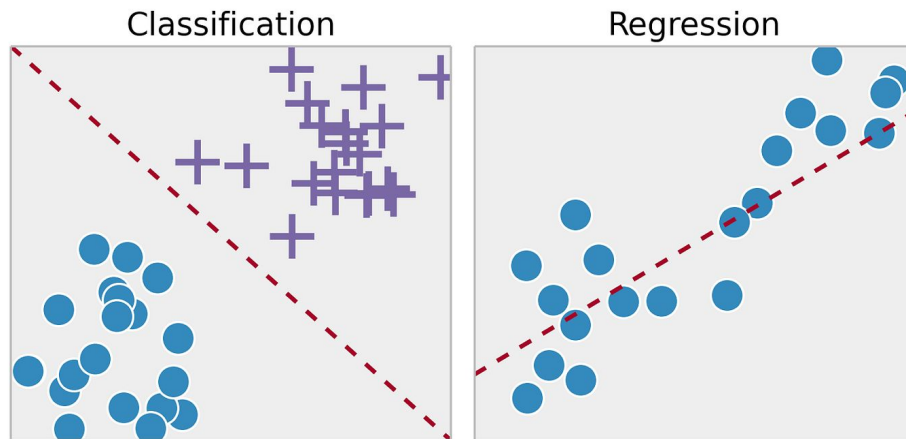
<https://deepmind.google/discover/blog/ai-solves-im-o-problems-at-silver-medal-level/>

Tipos de supervised learning

Nos vamos a concentrar bastante en el aprendizaje supervisado. En este tipo de aprendizaje, tenemos dos grandes categorías de problemas:

Regresión: buscamos una función que arroje un valor continuo.

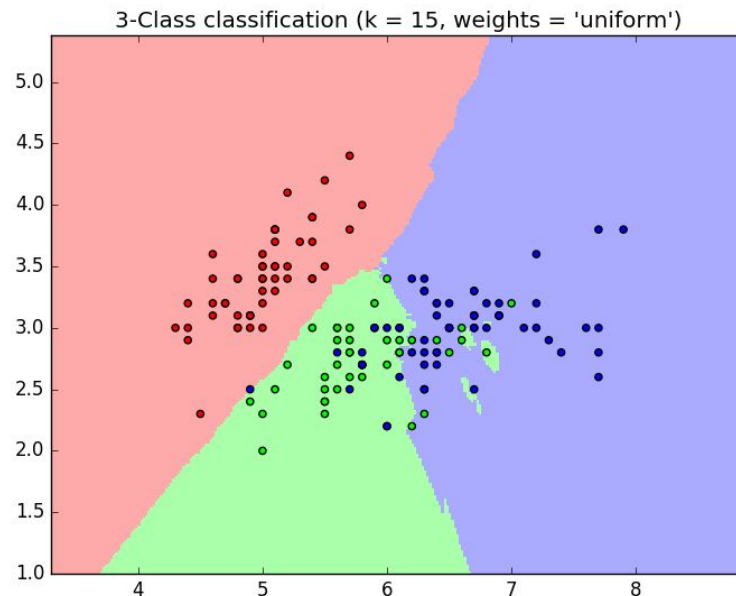
Clasificación: buscamos una función que asigne una categoría.



Clasificación – ¿Qué estamos haciendo?

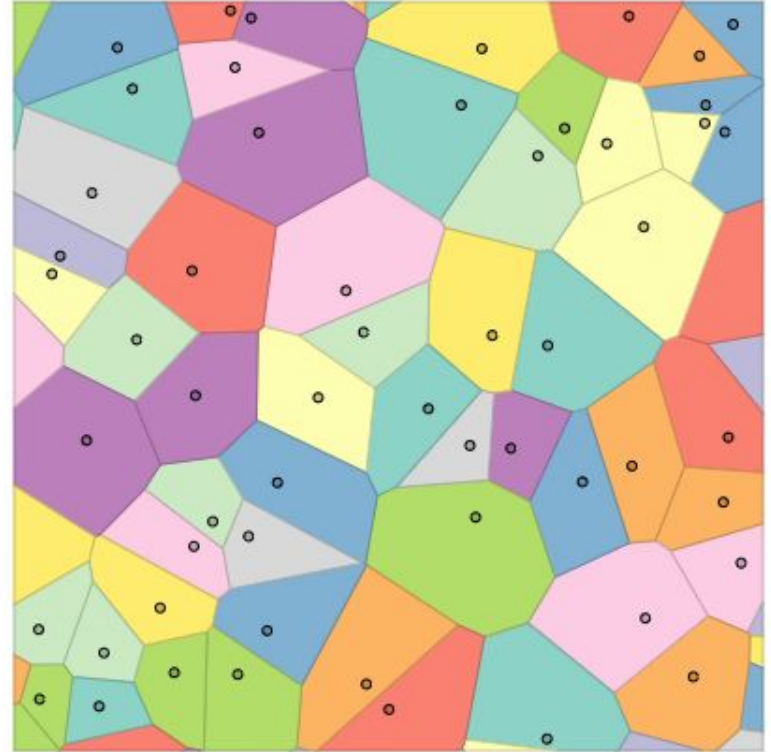
Lo que vamos a estar haciendo es definiendo *decision boundaries* que nos indican cómo se divide el espacio.

Parece “ideal” encontrar explícitamente esas *decision boundaries*. En general eso no va a suceder.



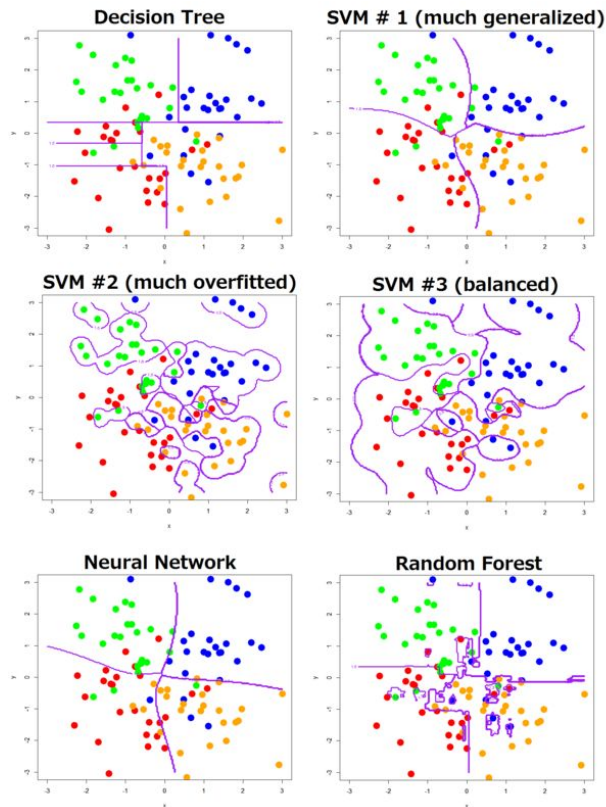
Decision Boundaries

En algunos casos muy particulares se podrían llegar a obtener exhaustivamente, pero es algo muy peculiar.



Decision boundaries in the wild

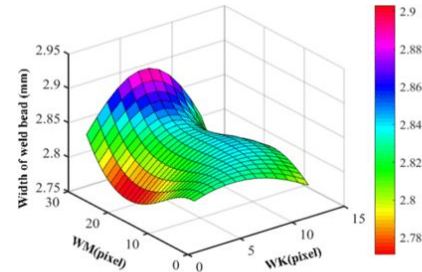
En los problemas reales estos límites son muy complejos, no lineales y están en altas dimensiones. Vamos a ir viendo hasta dónde nos permite llegar cada modelo.



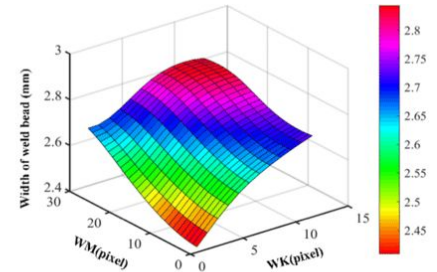
Regresiones

En el caso de las regresiones en vez de decisiones boundaries vamos a estar aproximando funciones.

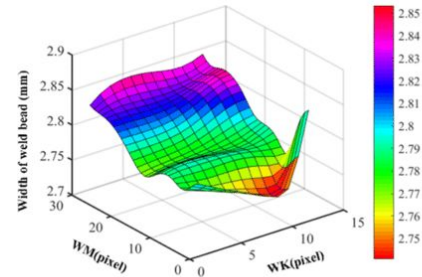
Nuevamente estas funciones estarán en altas dimensiones y serán muy complejas para poder explicar la naturaleza de los puntos.



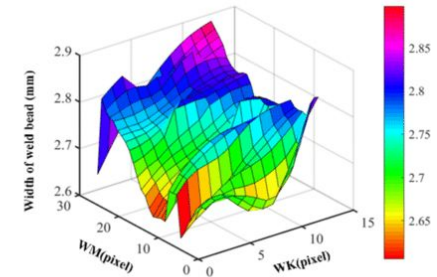
(a) 3D surface of RBFNN with LK=8 pixel and AK=80 pixel



(b) 3D surface of BPNN with LK=8 pixel and AK=80 pixel



(c) 3D surface of GRNN with LK=8 pixel and AK=80 pixel



(d) 3D surface of ENN with LK=8 pixel and AK=80 pixel

Problemas y modelos correctos

Etiquetar los problemas en general es muy útil, ordena y orienta, pero cuidado con no pasarse de largo. Algo que parece un tipo de problema por ahí se resuelve mejor con un modelo adecuado para otro tipo de problema. Una serie de tiempo a veces puede que sea mejor aproximada con un método de árboles como XGBoost.

El método correcto para el problema correcto: No Free Lunch Theorem*

*acá se separa la teoría de la práctica, en la práctica no es “free lunch” pero algunos modelos están cerca

¿Qué estamos haciendo? (Matemáticamente)

Una *muestra* es un dato del mundo real. Una muestra tiene m *features* o características.

Cuando pensamos en una muestra única, la pensamos como un vector columna, pero en realidad luego nos vamos a referir siempre a una muestra como una *fila*.

$$\begin{bmatrix} x_1 & x_2 & \dots & x_m \end{bmatrix} \\ = \\ \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix}$$

¿Qué estamos haciendo? (Matemáticamente)

Cuando tengamos muchas muestras las vamos a disponer como n filas y sus features serán m columnas. En aprendizaje supervisado, la columna a predecir es una más de la matriz o a veces se la diferencia como el vector y .

¿Cómo son las filas y columnas de los problemas que vimos hasta ahora?

$$\mathbf{X} = \begin{bmatrix} x_1^{[1]} & x_2^{[1]} & \dots & x_m^{[1]} \\ x_1^{[2]} & x_2^{[2]} & \dots & x_m^{[2]} \\ \vdots & \vdots & \ddots & \vdots \\ x_1^{[n]} & x_2^{[n]} & \dots & x_m^{[n]} \end{bmatrix} \quad \mathbf{X} = \begin{bmatrix} \mathbf{x}_1^T \\ \mathbf{x}_2^T \\ \vdots \\ \mathbf{x}_n^T \end{bmatrix}$$

¿Qué estamos haciendo? (Matemáticamente)

$$\mathbf{X} = \begin{bmatrix} x_1^{[1]} & x_2^{[1]} & \dots & x_m^{[1]} \\ x_1^{[2]} & x_2^{[2]} & \dots & x_m^{[2]} \\ \vdots & \vdots & \ddots & \vdots \\ x_1^{[n]} & x_2^{[n]} & \dots & x_m^{[n]} \end{bmatrix} \xrightarrow{f} \mathbf{y} = \begin{bmatrix} y^{[1]} \\ y^{[2]} \\ \vdots \\ y^{[n]} \end{bmatrix}$$

En los datos reales hay una función* f que relaciona las columnas/features/características de las muestras con la columna target/label/ y . Los métodos de Machine Learning entrenarán una función h que relacionará X con $\hat{y}$, buscando que $\hat{y}$ "se parezca" a y .

Buscando la “h”

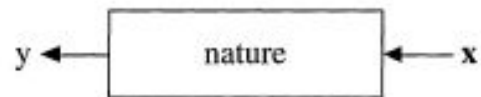
¿Qué necesitamos para buscar la función h ? (además de datos)

- Clases de algoritmos o modelos: Ya conocen algunos de Inferencia Estadística.
- Algoritmos de optimización para entrenar los modelos. Una vez que defino un modelo, ¿Cómo hago para encontrar la mejor instancia de ese modelo?
- Métricas para evaluar el entrenamiento: ¿Cómo mido qué tan bueno es mi modelo? ¿Qué impacto tiene usar diferentes métricas?

Las dos culturas

Pensando en problemas tabulares, ya sabíamos esto de conseguir una h para relacionar X con y . ¿Por qué no nos conformamos con regresiones lineales para regresiones, regresión logística para clasificaciones y listo?

Abstract. There are two cultures in the use of statistical modeling to reach conclusions from data. One assumes that the data are generated by a given stochastic data model. The other uses algorithmic models and treats the data mechanism as unknown. The statistical community has been committed to the almost exclusive use of data models. This commitment has led to irrelevant theory, questionable conclusions, and has kept statisticians from working on a large range of interesting current problems. Algorithmic modeling, both in theory and practice, has developed rapidly in fields outside statistics. It can be used both on large complex data sets and as a more accurate and informative alternative to data modeling on smaller data sets. If our goal as a field is to use data to solve problems, then we need to move away from exclusive dependence on data models and adopt a more diverse set of tools.



There are two goals in analyzing the data:

Prediction. To be able to predict what the responses are going to be to future input variables;

Information. To extract some information about how nature is associating the response variables to the input variables.

Statistical Modeling: The two cultures, Leo Breiman, Statistical Science, 2001

Las dos culturas

- Tenemos muchísimo más cómputo que hace 50 años, ¿Por qué quedarse con fórmulas “cerradas”?
- Tal vez queremos dar explicaciones simples y perfectas a procesos que no las tienen. Tal vez el proceso ya es muy complejo.
- Los métodos de Machine Learning son interpretables!

Tabular vs non-tabular ML

¿Y las aplicaciones de imágenes? ¿Los textos? ¿Los audios? ¿Las IA generativas? ¿Dónde está todo eso?

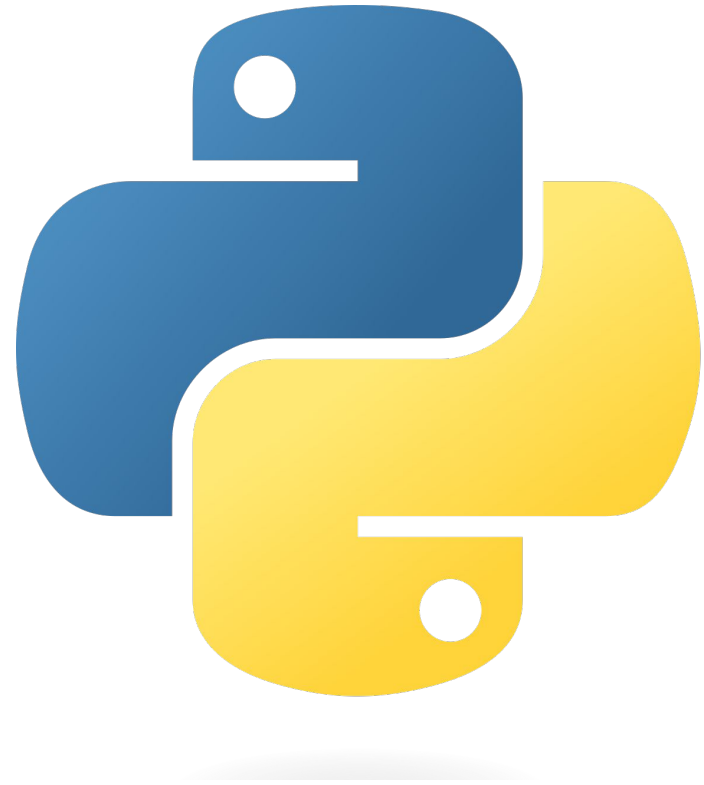
Los problemas vistos son del tipo “tabular ML” o “structured ML”, ¿Por qué?

El “Business ML” va mucho por este lado.

En realidad también sirve para data no estructurada o problemas “non-tabular”.

Es donde todavía el Machine (non-deep) Learning tiene lugar.

Muy pintorescos todos los problemas que podemos resolver pero para resolverlos en serio necesitamos programar (por ahora)



Bienvenido chatGPT (o Gemini, o Llama, o Claude, o el que sea). Pasen, los estabamos esperando, pero con cuidado...



Programación

Worldwide, Mar 2026 :

Rank	Change	Language	Share	1-year trend
1		Python	34.87 %	+4.4 %
2	↑↑	C/C++	13.66 %	+6.6 %
3	↓	Java	9.82 %	-5.4 %
4	↑↑	R	6.49 %	+1.9 %
5	↓↓↓	JavaScript	4.9 %	-3.2 %
6	↑↑↑↑↑↑	Swift	3.5 %	+1.1 %
7	↑	Rust	3.08 %	-0.0 %
8	↓↓↓	C#	3.03 %	-3.1 %
9	↓↓	PHP	2.9 %	-0.8 %
10	↑↑↑↑↑↑	Ada	2.66 %	+1.3 %

<https://pypl.github.io/PYPL.html>

Programación en Data Science

Las competencias de Data Science no son un reflejo exacto del mundo industrial, pero son una buena aproximación.

<https://mlcontests.com/state-of-competitive-machine-learning-2024/>

Winning Solutions

To track tools and techniques used by competition winners, we analysed the #1-ranked solution for as many of the competitions in 2025 as we could,¹³ and gathered information on programming languages, packages, model architectures, and computational resources.

Almost all winning teams used Python as the main programming language for their solutions, in line with previous years. The one exception we found was Kaggle's [Efficient Chess](#) competition, for which the winner used C as their primary language, alongside Rust and Python.

The packages listed below represent the key Python packages that make up competition winners' toolkit.

Python Packages

Core

numpy	
scipy	
scikit-learn	# models, transforms, metrics
matplotlib	# low-level plotting
seaborn	# higher-level plotting

NLP

transformers	# tools for pre-trained models
peft	# parameter-efficient fine-tuning
trl	# reinforcement learning
unsloth	# efficient training/inference 
vllm	# efficient inference 
sentence-transformers	# embedding models

El reporte tiene muchos más datos interesantes. Por ejemplo, tiene estadísticas sobre los modelos utilizados y las metodologías.

DataFrames

DataFrames remain a two-bear race: 61 of the analysed winning solutions used Pandas, and 5 used Polars.

Of the 5 that used Polars, two used it minimally for IO, two used a mix of Polars and Pandas (e.g. some team members used Pandas, some Polars), and [one](#) used Polars as its primary DataFrame library with Pandas only for interfacing with other machine learning libraries.

For more analysis of Polars use in solutions, see the [2024](#) and [2023](#) editions of this report.

Tabular Data

Gradient-boosted decision trees (GBDTs) remain the go-to tabular competition winner's modelling tool, sometimes as part of an ensemble alongside neural nets.

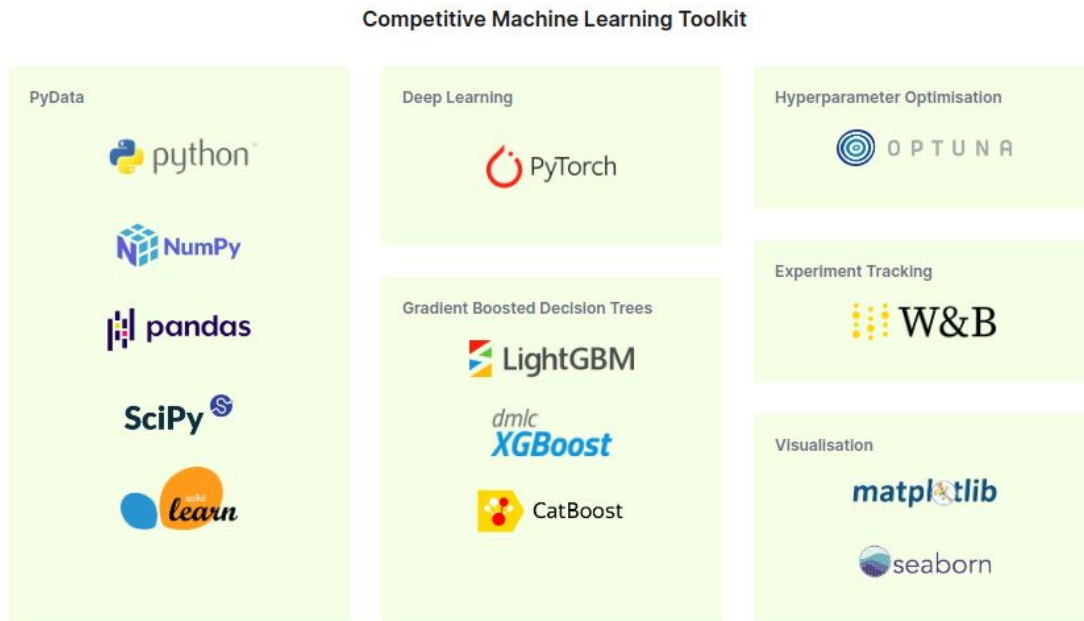
Among winning solutions that used GBDTs, XGBoost and LightGBM were most popular this year, with 14 uses each. We found 8 winning solutions using CatBoost. Although part of the choice of library is likely due to familiarity, some winners explicitly stated that they experimented with all three of these libraries, and found significant performance differences between the different libraries on their dataset.¹⁵

[AutoGluon](#) was used for tabular prediction in two winning solutions. Most notably, the winner of Kaggle's Open Polymer Prediction 2025 competition used AutoGluon as part of an ensemble, and noted that AutoGluon "was able to beat an ensemble of XGBoost, LightGBM, and TabM models [tuned with] Optuna", while using only a fraction of the training budget.¹⁶

For the first time, we also saw a tabular foundation model (TabPFN) used, by the [winners](#) of DrivenData's PREPARE challenge. The winning team used TabPFN to generate features that were fed into GBDT models.

We also found [one winning solution](#) using the TabM tabular deep learning model to predict transplant survival rates in a Kaggle competition. Their solution used TabM as part of an ensemble of models, including GBDTs and a combination of k-nearest-neighbours and graph neural nets.

Dentro del mundo Python hay algunas librerías muy utilizadas en la práctica (no son las únicas!)



Stack DS – Virtual Envs

Es **MUY** recomendable que utilicemos Python en ambientes virtuales para instalar diferentes paquetes.

Si usan Anaconda:

<https://docs.anaconda.com/free/navigator/tutorials/manage-environments/>

Si usan Python de otra manera:

<https://docs.python.org/3/library/venv.html>

